

PART 2 - Computer Science 2 (CS2)

Chapter 10

CS2 course package - CDER

John Smith¹ and Jane Doe²

¹JTA University

²XYZ University

Abstract

|Your abstract text here.|

Descriptor Elements

Content Overview: |e.g., programming II and object-oriented programming|

Programming Language Employed: |e.g., Java, others C/C++|

Relevant core courses: |e.g., CS1, Data Structures|

Pre-requisites: |Required student background|

Relevant PDC topics: |List topics here with bloom classification (use K-knowledge, C-comprehension, or A-be able to apply)|

Relevant PDC topics and Blooms levels

PDC Concept	Chapter Section				
	1.1	1.2	1.3	1.4	1.5
Topic1	K	C	A		
Topic2	K	C	A		
Topic3		C	A	A	

Learning outcomes:

1. |Outcome 1|
2. |Outcome 2|

Context for use: |Your local context about institution, department, mix and levels of students, class size, language employed, etc.|

10.1 Introduction

|Motivation and background for the chapter.|

10.2 How CS1 was changed and PDC topics integrated

|Explain how the existing course (e.g., CS1) was modified.|

10.2.1 Integrated Syllabi (Pre and Post)

|Provide high level syllabi highlighting changes.|

10.2.2 Highlights

|What remained the same, and what specifically needed to be changed or deleted?|

10.2.3 Challenges

|Discuss logistical, pedagogical, or technical difficulties.|

10.3 New Unplugged Activities

10.3.1 Overly Complicated Machine ([Srishti will write this.](#))

Problem Description, Prerequisites, and Learning Outcomes

Implementation Details

- Logistics (Material required for the activity, etc.)
- How to Conduct
- Instructional materiel (Lectures, Videos, Assignments, etc.)

Experience and Teaching Tips

Data and Analysis

10.4 New Plugged-in Activities

10.4.1 Fork Join Activity ([David will write this.](#))

Problem Description, Prerequisites, and Learning Outcomes

Implementation Details

- Logistics (Material required for the activity, etc.)

- How to Conduct
- Instructional materiel (Lectures, Videos, Assignments, etc.)

Experience and Teaching Tips

Data and Analysis

10.4.2 Parallel vs. Serial Sort Algorithm Comparison (April will write this.)

Problem Description, Prerequisites, and Learning Outcomes

Algorithms

Implementation Details

- Instructional Material (Lectures, Videos, Assignments, Tests)
- How-To Guides

Experience and Teaching Tips

Data and Analysis

Example Implementations

The following institutions are using a variation of this assignment and you can find their details in their specific chapters:

- TNTECH - CS1 course - Section 2.4.1
- UNL - CS1 course Section 14.4.2

10.4.3 Parallel Image Processing

Activity Summary

The parallel image design is meant for higher-education CS departments that want an instructor-ready activity with both conceptual scaffolding and implementation detail[?]. The plan follows the lecture-demo-lab structure, uses the provided Java multithreading slide deck as the conceptual guide, and translates the parallel image blur lab into a fully scaffolded CS2 lesson.

Pedagogically, the strongest reason to use image processing for CS2 concurrency is that it is both inherently data parallel and visually self-explanatory. The pixel-level operations make abstract concurrency ideas tangible because students can see both performance gains and data corruption in the resulting images; it also recommends a combined sequence of lecture, demonstration, and experiment rather than lecture alone. The provided slide deck introduces parallelism, concurrency, race conditions, synchronization, speedup, overhead, `ExecutorService`, and mathematics for various image-processing kernels.

For a default implementation, this plan assumes at least two class meetings totaling about 135 minutes: one concept-and-demo session and one guided lab session. If your department runs longer lecture, the same material can be collapsed into a single 115 minute workshop by shortening discussion and moving the debrief outside class.

Executive Summary

Students begin with a short unplugged partitioning warm-up inspired by FlagMake in CS1, a course they took in the previous semester, then a conceptual lecture to present the terminologies in PDC, code explanation on edge detection and histogram generation. Later, students observe a web browser-based multithreaded image-processing demonstration with multiple different kernels such as RGB to grayscale, edge detection, and blur using various numbers of threads. The speedup is automatically calculated and displayed to students. Then, students are asked to implement two multi-threaded Java programs for box blur lab: a sequential baseline, a parallel row-partitioned version. They measure runtime across thread counts, compare outputs, calculate speedup by hand, and discuss why more threads do not always produce faster execution. The learning target is not merely “use threads,” but to understand the trade-offs among correctness, overhead, scalability, and code complexity.

Example Implementations

The following institutions are using a variation of this assignment and you can find their details in their specific chapters:

- MSU - CS2 course - Section
- USI - CS2 course Section
- HPU - CS2 course section ??

Student Learning Outcomes

By the end of this activity, students should be able to:

- Explain the difference between sequential execution, concurrency, and parallelism; use Java multithreading tools to implement a row-partitioned image kernel.
- Analyze the trade-off between speedup and overhead; distinguish embarrassingly parallel work from work requiring synchronization.
- Apply synchronization mechanisms when necessary.

Conceptual objectives for students are these:

- Distinguish parallelism from concurrency and describe the role of a thread versus a processor/core.
- Define race condition, critical section, synchronization, speedup, and parallel overhead.

- Predict why runtime improvement may plateau or reverse when thread count becomes too large.

Practical objectives for students are these:

- Read an image, traverse pixels, and compute a 3×3 box blur.
- Use `ExecutorService` to submit tasks to a fixed thread pool. The official JDK documentation defines `ExecutorService` as an executor that manages asynchronous tasks and provides orderly shutdown methods.
- Use `CountDownLatch` so the main thread waits until worker tasks finish.
- Access pixels with `BufferedImage.getRGB()` and `BufferedImage.setRGB()`, and read/write image files with `ImageIO.read()` and `ImageIO.write()`. Create two image buffers as read-only input buffer and non-conflicting output buffer to ensure data correctness without synchronization mechanism.
- Collect timing data, compute speedup, and compare sequential, unsynchronized parallel, and synchronized parallel variants.

Prerequisite

This lab is best placed in CS2, after students are comfortable with loops, arrays, methods, objects, file I/O, and basic exception handling. The Java lab assumes JDK installation and basic Java familiarity. A short pre-lab warm-up is therefore recommended: have students mark a 6×6 pixel grid and identify which cells one worker, two workers, and four workers would compute. This preserves the “students take the role of processors” spirit of FlagMake while leading naturally into row partitioning for the lab. Students do not need a prior course in operating systems, architecture, or high-performance computing.

Algorithms

The Basic Idea: Convolution

A grayscale image is a grid of numbers: $I(x, y)$ is the brightness of the pixel at column x , row y ($0 = \text{black}$, $255 = \text{white}$).

Blurring operation slides a small 3×3 matrix K (called a kernel) over the image. At each pixel, multiply each kernel entry by the pixel under it and add everything up:

$$I'(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 K(i, j) I(x + i, y + j). \quad (10.1)$$

Blurring replace each pixel with the **average of itself and its 8 neighbors**. In kernel form:

$$K_{\text{box}} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}. \quad (10.2)$$

Every neighbor counts equally. The weight of each pixel is the same, so the overall brightness of the image does not change. The blurred pixel is:

$$I'(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 I(x + i, y + j). \quad (10.3)$$

Example. If the 9 pixels under the kernel are 10, 10, 10, 10, 10, 200, 10, 200, 200, the new center value is their average:

$$\frac{10 + 10 + 10 + 10 + 10 + 200 + 10 + 200 + 200}{9} = \frac{660}{9} \approx 73.$$

A bigger kernel (e.g. 5×5 with $\frac{1}{25}$) averages over more neighbors and gives a stronger blur.

Materials and Running Environment

The original blur lab specifies JDK 8+, `images/input/` and `images/output/` folders, and an example image named `aroma.jpg`. It also expects two Java files: a sequential baseline, and a parallel skeleton. Use Table 10.2 before class to reduce troubleshooting time and keep the lab focused on multithreaded computing rather than configuration problems.

Table 10.1: Instructor Resources.

Resource	Purpose
Lecture slides: <code>class10_PDC.pptx</code>	Introduce sequential vs. parallel thinking, overhead, speedup, race conditions, synchronization, and the image-processing example.
Student handout: adapted <code>CSIT112 Lab5_PDC.pdf</code>	Guides the blur implementation, experiments, and report.
Starter code set	<code>BoxBlurSeq.java</code> , <code>BoxBlurParallel.java</code> ,
Performance worksheet	A simple table for thread count, runtime, and speedup.

Note: For beginners, distribute instructor-curated images rather than asking students to search for their own during lab time. This keeps the lesson focused on multithreading rather than file wrangling and reduces licensing ambiguity. If students later use personal images for extension work, require them to note the source and confirm reuse permissions.

Instructor Notes and Teaching Tips

The most important misconception to surface early is that concurrency and parallelism are not the same thing. The slide deck distinguishes concurrency as multiple tasks making progress and parallelism as simultaneous execution on multiple cores. Students often overuse these words interchangeably; correcting that vocabulary early makes later performance discussions much cleaner.

A second misconception is that “more threads always means faster.” The runtime table in the slides shows improvement through a small number of threads and then worse performance as thread count rises, illustrating setup cost, scheduling cost, and synchronization cost. This is the right moment to introduce a practical form of Amdahl’s Law: the useful work is not the only work the machine performs.

A third misconception is that “One Java statement executes atomically, thus no need to protect the update of every brightness bin for the parallel histogram program as in the *synchronized (lock) hist[v]++;* . We need to explain that a single Java statement is divided into multiple separate instructions with the read and write separated by a gap time.

A fourth misconception is that one-task-per-row is always the best partitioning strategy. The original lab asks students to try multiple rows per task as an experiment. Make that extension visible from the start: row-level tasks are easy to teach, but chunked rows often reduce scheduling overhead and become a natural segue into load balancing and granularity.

Lesson Plan

Table 10.2: Setup Checklist.

Setup item	Instructor action	Student verification	Troubleshooting notes
Java installation	Confirm lab machines or student laptops have JDK 8+; use the department standard JDK if available.	Run <code>java -version</code> and <code>javac -version</code> .	If <code>javac</code> is missing, students may have a JRE rather than a JDK or their <code>PATH</code> may be incorrect.
Project structure	Distribute a zip or repository with <code>src/</code> , <code>images/input/</code> , and <code>images/output/</code> .	Open the project and confirm the folders exist.	Many file-not-found errors come from running the program from the wrong working directory.
Input image	Place <code>aroma.jpg</code> or a curated image in <code>images/input/</code> .	Confirm the image opens outside Java and the filename matches the code exactly.	Case-sensitive systems distinguish <code>Aroma.jpg</code> from <code>aroma.jpg</code> .
Starter code	Check that <code>BoxBlurSeq.java</code> compiles.	Compile and run the sequential file first.	Do not let students debug threading before the sequential works.
IDE or terminal workflow	Decide whether students will use an IDE, terminal commands, or both.	Record how they compile and run programs for the report.	Pay attention to the changed working directories and output path.
Timing method	Require <code>System.nanoTime()</code> or provide timing code.	Use the same code-block for timing and same image.	Do not include image loading or writing unless you intentionally measures end-to-end time.
Thread-count test set	Post the required thread counts: 1, 2, 4, 8, 16, and 32, or adjust for local hardware.	Record all required thread counts in the lab.	For machines with few cores, high thread counts are still useful for showing overhead.
Output verification	Show students the expected output filenames and where to find them.	Open each output image after running the program.	A program that prints a time but writes no output image is not complete.

Table 10.3: Timing Plan for a Two-Session Implementation.

Phase	Time	Instructor actions	Student actions	Checkpoint
Session 1: Warm-up	5–10 min	Introduce the goal and run a small grid partitioning.	Predict how one, two, and four workers divide pixels.	Completed partitioning sketch.
Session 1: Concept lecture	20–25 min	Present sequential, parallelism, threads, cores, race conditions, synchronization, speedup.	Multiple choices questions or discussion.	Knowledge of PDC vocabulary.
Session 1: Image-processing demo for edge detection and histogram	20–25 min	Demonstrate kernels, thread counts, output comparison, and non-linear performance.	performance and correctness.	Completed pre-lab prediction table.
Session 1: Code orientation	15–20 min	Walk through file structure, starter code, input/output folders, and timing method.	Compile the sequential file and confirm running environment.	Environment check done.
Session 2: Sequential baseline	10–15 min	Review the 3×3 blur and border handling.	Run <code>BoxBlurSeq</code> and save timing.	Sequential runtime and output image.
Session 2: Parallel version	20–25 min	Guide students through thread pool, row tasks, and <code>CountDownLatch</code> .	Complete <code>BoxBlurParallel</code> and test thread counts.	Parallel output and timing.
Session 2: Experiments	15–20 min	Prompt students to vary thread counts and chunk sizes.	Compute speedup.	Completed performance worksheet.
Session 2: Debrief	10–15 min	Facilitate discussion using the post-lab debrief artifact.	Submit lab reflection.	Lab reflection.

Bibliography

Appendix

|You must include these four specific subsections (in addition to anything else):|

10..4 Github Link to Instructional Material

`https://github.com/...`

10..5 Detailed Pre and Post Syllabus

10..6 Organization of Material

|Explain the folder structure of your Github repository.|

10..7 Survey and Other Anonymized Data and Analysis

Development and testing teams administered the following pre- and post-course survey which asked students to rate their “comfort level” with 19 computing and 4 PDC-related concepts on the following on a 7-point Likert scale:

1. No Experience At All
2. Very Little Experience
3. Slightly Experienced
4. Between Slightly/Moderately Experienced
5. Moderately Experienced
6. Very Experienced
7. Extremely Experienced

Specifically:

1. Classes & Objects
2. List data structures
3. Generics (parameterized polymorphism)
4. Stacks
5. Queues
6. Sorting Algorithms
7. Comparators

8. Recursion
9. Maps
10. Sets
11. Trees
12. Binary Search Trees
13. Priority Queues
14. Hashing
15. Graphs
16. Writing Unit Tests & Test Cases
17. Big-O Notation
18. Choosing the correct data structures to solve a problem
19. Designing & Writing Classes

as well as the following PDC concepts:

20. Problem Decomposition: Breaking the desired functionality into separate parts (e.g. methods) that can be implemented separately
21. Parallel Processing: Computing where the operations within a program are being executed simultaneously (i.e. running on different cores of the processor at the same time)
22. Distributed Computing: Computing with a program that runs using data from a different computer (i.e. an online service)
23. Event handling: Having a function that responds to an independent action (i.e. a mouse click)

Student attitudes were measured by asking students to agree/disagree with the following 5 statements

1. In general, my interest in computing is high
2. My interest in learning about parallel and distributed computing is high
3. In general, I believe learning about parallel and distributed computing is important
4. I believe I have a good understanding of computing
5. I believe I have a good understanding of parallel and distributed computing

The following 7-point Likert scale was used:

1. Strongly Disagree
2. Disagree
3. Somewhat Disagree
4. Neutral
5. Somewhat Agree
6. Agree
7. Strongly Agree